

FIU SENIOR DESIGN · TEAM 18

Biometric Steering Wheel Multi-PPG Implementation

Architecture · Firmware · Verification · Operations

Team 18 · FIU Senior Design

Prepared October 4, 2026 · Branch `samantha/multi-ppg`

Firmware implementation snapshot: `49918f10384f903df1c0768757f4bf7168f55ddc`

Documentation edition: consolidated technical guide · Navy edition · October 4, 2026.

Branch scope: `samantha/multi-ppg` only. This is the current guide for this branch. It replaces the earlier multi-PPG edition; it does not redefine the implementation on `LuisFullSystem`, `main`, or other branches.

This guide documents the ESP32 extension from one PPG sensor to four selectable PPG sensors. It explains every file in the implementation commit, what changed, how to run it on Fedora, and what the tests do and do not establish. It accompanies Luis's **Team18_System_Guide.pdf** and its repository source, `docs/SYSTEM_GUIDE.md`.

Normal operation is standalone. The ESP32 acquisition, sensor-selection, and vitals firmware is stored in the ESP32's nonvolatile flash. Once uploaded, it starts automatically whenever the board receives power or resets. A laptop, VS Code, terminal commands, serial monitor, and Fedora USB permissions are not runtime requirements. Development and flashing use Luis's existing terminal workflow; no editor-specific task configuration is required.

Current status: four-sensor discovery, single-source selection, shutdown commands, and contact-driven switching have been exercised. The existing packet format is preserved. Measurement accuracy remains unvalidated, and live ESP32 → Pi → phone delivery has not been verified in this work session.

Four optical sources. One selected data stream.
Standalone ESP32 operation with the existing Pi interface.

Contents

- 1. How this work fits into Luis's system 3
- 2. Hardware and the meaning of “one sensor” 6
- 3. Runtime behavior from startup to a packet 7
- 4. Current implementation files and their responsibilities 9
- 5. What the Pi receives and what it does not..... 13
- 6. Development computer setup and optional trial tools 14
- 7. Reading diagnostics and collecting a waveform..... 15
- 8. Problems encountered, actions taken, and remaining limits..... 18
- 9. Verification and an accuracy plan 19
- 10. Commit history and the next ecosystem phase..... 21
- 11. Source map and document maintenance..... 22

1. How this work fits into Luis's system

The intended ecosystem is:



The diagram describes the intended data path, not proof that every link was tested. Luis's guide documents the Pi relay, phone application, database, and website. Our extension changes the source selection inside the ESP32; downstream components continue to receive the same logical stream.

Git ancestry was checked: Luis's team-repository commit `f2c6b3c` is the parent of Samantha's `e01b611` ("Add multi-PPG sensor failover"). Samantha's change touched only `system/esp32/ppg_transmitter/ppg_transmitter.ino`. Our subsequent commit `49918f1` adds the selection refinements, diagnostics, supporting tools, tests, and notes described here.

The Pi, mobile app, website, shared protocol files, `ppg_frame.h`, and `ppg_vitals.h` were compared with `origin/LuisFullSystem` and remained unchanged at this snapshot. Luis's estimator was retained; we did not replace it with the Maxim comparison algorithm.

Repository map: what runs where

The folders separate deployment targets. They are not all flashed onto the ESP32. The following map covers the ecosystem folders and the files directly involved in acquisition, transport, tests, and documentation. Luis's original guide provides the full mobile/database implementation reference.

Folder	Responsibility	Runs on / deployment
system/esp32/	Acquisition, MUX selection, packet production and host upload tools	Sketch/headers compile to ESP32 firmware; scripts run on a development computer
system/common/	Master Python packet decoder and Bluetooth link helpers	Shared source; copied into Pi and laptop packages
system/pi/	Receive ESP32 BLE packets, validate/log them, and relay to phone	Raspberry Pi, using the existing service installer
system/laptop/	Optional graphical waveform/viewer and direct BLE bench connection	Development/bench computer; not required in the vehicle

Folder	Responsibility	Runs on / deployment
system/tests/	Protocol, estimator and simulated MUX regression checks	Development computer or CI
mobile-app/	Phone UI, BLE client, drive storage, analysis, voice interaction and sync	Phone build; development/build tools run elsewhere
mobile-app/supabase/	Database schema, migrations and application scripts	SQL applied to the configured Supabase project
website/	Web dashboard consuming stored/live cloud data	Web hosting and a browser
docs/	Human-readable system and multi-PPG guides, hardware documents	Documentation, not runtime code
.github/workflows/	Repository automation, including Luis's iOS build workflow	GitHub Actions when configured/triggered

ESP32 and transport file reference

File	Detailed role
system/esp32/ppg_transmitter/ppg_transmitter.ino	Owns initialization, sensor state, buffering, estimator calls and BLE notifications
system/esp32/ppg_transmitter/ppg_vitals.h	Pure C++ pulse estimator: baseline removal, normalized autocorrelation, rate selection and red/IR ratio calculation; retained from Luis
system/esp32/ppg_transmitter/ppg_frame.h	Packs fields and paired 18-bit samples into the fixed protocol layout and appends CRC; retained from Luis
system/esp32/flash_esp32.sh	Host-side compile/upload wrapper using Arduino CLI and 40 MHz DIO settings; not executed on the microcontroller
system/esp32/decode_serial_capture.py	Offline diagnostic packet-to-CSV converter; no sensor-control responsibility
system/esp32/README.md	Current terminal quick-start and link to this guide
system/common/ppg_protocol.py	Python frame decoder, CRC checks, reassembly and shared packet constants
system/common/bluez_links.py	Linux BlueZ connection-management helpers
system/pi/ppg_relay.py	Scans/connects to ESP32, subscribes to notifications, reassembles packets and serves the phone-side relay
system/pi/ppg_monitor.py and ppg-monitor	Pi status display and launcher; separate from the relay's transport logic
system/pi/setup_pi.sh	Installs/configures the existing Pi environment and boot service

File	Detailed role
system/pi/wheels/	Bundled Python dependencies for the documented Pi installation; not firmware
system/pi/ppg_protocol.py and bluez_links.py	Deployment copies of the shared Python modules; tests check consistency
system/laptop/ppg_viewer.py and run_viewer.sh	Optional desktop viewer and launcher; their shared-module copies follow the same consistency rule
system/tests/test_protocol.py	Python protocol/reassembly checks and host C++ checks, including decoder/encoder compatibility
system/tests/test_frame_host.cpp	Exercises C++ packet encoding on the computer so the Python decoder can verify it

Phone, database and web responsibilities

Within `mobile-app/`, `components/` presents the interface; `lib/ble/` handles phone BLE and protocol interpretation; `lib/hooks/useDriveSession.ts` connects received frames to the drive workflow; `lib/db/` handles local records and synchronization; `lib/archive/` manages raw-data archives; `lib/analysis/` contains the existing baseline/flag logic; and `lib/voice/` implements voice-check behavior. These are inherited components, not rewritten for four PPGs.

`mobile-app/supabase/apply_all.sql` assembles Luis's migrations. `revert_all.sql` and `legacy_samantha.sql` have separate, documented purposes and should not be run merely to enable the MUX. No database migration was needed for this source-selection change. The website consumes the existing cloud representation; it receives no new four-channel packet structure.

2. Hardware and the meaning of “one sensor”

Component	Role and configuration
ESP32	Controls acquisition, estimates vitals, builds packets, advertises over BLE
TCA9548A	I2C multiplexer at address 0x70; selects a downstream bus
PPG1-PPG4	MAX30102 modules at address 0x57, on MUX channels 0-3
ESP32 I2C pins	SDA GPIO21; SCL GPIO22
USB / CP210x	Powers the connected board as wired, uploads firmware, and carries serial diagnostics
Raspberry Pi	Intended BLE receiver and relay; not the USB serial monitor

All modules use the same sensor address. Selecting only one MUX channel avoids address collisions. A channel mask of `1 << channel` enables one downstream bus at a time.

The MUX does not switch the modules' power rails. Disconnecting a channel from I2C does not by itself stop that sensor's LEDs or ADC. The firmware explicitly commands sensor shutdown and reads back the shutdown bit. The modules can remain electrically powered while idle. Use the actual breakout modules' voltage requirements and a common ground; software does not establish power-supply adequacy.

“One source” means one selected module supplies all red/IR samples, BPM, and SpO2 for a given frame. It does not mean sending just one scalar: Luis's packet already contains 100 paired raw samples plus derived values and flags.

3. Runtime behavior from startup to a packet

Power on is the normal start command

```
Power the ESP32 -> stored firmware starts -> setup() initializes hardware
                -> loop() searches, reads, estimates, and serves BLE data
```

This follows Luis's existing Arduino startup design. `setup()` executes once per boot; `loop()` repeats automatically. There is no requirement to click Run in an editor, issue a serial command, or keep the development computer connected. Serial printing is diagnostic; the sketch does not wait for a serial monitor to be opened.

Flash again only when installing changed firmware. For ordinary use, supply suitable power to the assembled hardware. A computer USB cable supplied power during the trial, but a computer is not the intended runtime controller. Use the power arrangement documented by Luis and appropriate to the actual hardware.

Startup is automatic, not instantaneous: this sketch includes a brief boot delay and requires all four sensors to initialize before BLE advertising starts. If initialization fails, it retries automatically. After contact selection, signal history must accumulate before vitals can be accepted.

The complete ecosystem remains distributed as Luis designed it: acquisition and estimation run on the ESP32; relay functions run on the Pi; app, storage, and cloud functions run on their respective platforms. Only the ESP32 portion is flashed to the ESP32. Pi service startup and phone connection behavior will be verified in the next setup phase.

Automatic acquisition sequence

1. **Initialize:** start I2C; configure each of the four modules; verify shutdown before configuring the next. All four must initialize before normal operation starts. Failed startup attempts retry.
2. **Advertise:** start the existing BLE service as `SteeringWheelESP32`.
3. **Search:** begin at PPG1 and probe sequentially. Wake one module, allow 100 ms to settle, clear its FIFO, and read 20 fresh samples.
4. **Select:** require at least 16 of the 20 samples above the existing IR threshold of 100,000 and an adequate mean IR. The first passing module stays active; search stops.
5. **Acquire:** drain paired red/IR samples directly from that module's FIFO. Collect 100 raw pairs per frame at the configured 100 Hz rate.
6. **Estimate:** average groups of four samples to 25 Hz. Attempt vitals after 75 clean history samples, approximately three seconds; history grows to 200 samples, approximately eight seconds.
7. **Transmit when connected:** encode a protocol-v3 packet and notify the subscribed BLE central using the negotiated MTU.
8. **Recover:** after 50 consecutive low-contact raw samples, approximately 500 ms, discard the partial frame and history and search starting after the previous selected channel.

With no contact, each rejected probe is shut down; all modules sleep during the 250 ms pause between scans. During active acquisition, other modules are not repeatedly probed.

Contact is not pulse quality. Selection uses optical contact, not a verified heartbeat. If the selected module still detects contact but the estimator rejects its waveform, that module remains selected. Automatic switching solely because of low pulse quality is not implemented.

A brief below-threshold interruption also clears estimator history, even if it is too short to trigger channel switching. FIFO timeouts discard partial frames. Detected overflows discard the current frame and rebuild history. Failed shutdown verification stops the search before another sensor is awakened. A physically stuck bus may still require hardware intervention.

4. Current implementation files and their responsibilities

The files below describe the current branch. Firmware changes originate in commit 49918f1; subsequent workflow and documentation changes are incorporated into this edition.

4.1 system/esp32/flash_esp32.sh — terminal compile and upload

Luis's unchanged shell script is the shared upload entry point. It locates Arduino CLI, accepts a serial port or detects one, checks access, compiles for the classic ESP32 with 40 MHz DIO flash settings, and uploads the firmware. No editor-specific task file is included or required.

Run these from the repository root in any terminal, with Arduino CLI installed and serial access configured:

```
# Compile and upload when installing or changing firmware:
bash system/esp32/flash_esp32.sh /dev/ttyUSB0

# Optional: watch the running firmware after the upload finishes:
~/local/bin/arduino-cli monitor -p /dev/ttyUSB0 -c baudrate=115200
```

Close any existing serial monitor before uploading or opening a new monitor. Substitute the actual port if it differs. The upload script can also auto-detect a port when none is specified; with multiple devices, supply the intended port explicitly. Section 6 explains optional log capture with `tee` and `tail`.

Normal operation only requires powering the flashed ESP32. Neither these development commands nor an editor must remain running.

4.2 system/esp32/ppg_transmitter/ppg_transmitter.ino — modified firmware

This is the Arduino sketch compiled and uploaded to the ESP32. Samantha had already added MUX selection, four-channel initialization, verified sleep/wake operations, and multi-sensor failover. Our changes refine that implementation:

Change from Samantha's commit	Purpose
First passing contact wins	Stops scanning immediately instead of scanning all candidates and choosing the highest mean IR
Startup searches from PPG1	Replaces immediately streaming PPG1 before establishing contact
Search retry 2,000 → 250 ms	Reduces the idle pause between scans
First estimate at 75 rather than 100 history samples	Allows an attempt after two warm-up frames, around three seconds; not a guarantee of valid output

Change from Samantha's commit	Purpose
Immediate partial-frame discard on sustained contact loss	Avoids completing a frame after the loss threshold has been reached
Clear history on brief interruption	Prevents older contact data influencing the next estimate
Discard overflowed frames	Avoids treating a discontinuous window as intact data
Serial state, reason, history and peak correlation	Explains why readings are unavailable and which module is selected
Display invalid values as -- on USB	Improves readability without changing numeric packet values
Optional PPG_SERIAL_RAW build flag	Mirrors encoded packets as hexadecimal USB lines for waveform diagnosis; default is off

Key functions and data:

- `selectMux()` writes the single-channel mask.
- `setSensorAwake()` sends wake/shutdown and verifies the mode register.
- `sleepAllSensors()` attempts verified shutdown on every channel.
- `searchSensors()` performs sequential probes and selects one source.
- `resetSignalState()` clears FIFO caches, sample arrays, and estimator history. It does not reset the frame sequence counter.
- `drainFifo()` / `readRawSample()` supply ordered paired samples using the sensor FIFO directly.
- `loop()` builds raw and decimated data, checks contact and overflow, estimates vitals, and constructs the packet.
- `sendFrame()` sends MTU-sized notifications only when a BLE client is connected.

The selected IDE lines are inherited state:

```
uint32_t irBuffer[kAlgoLen];
uint32_t redBuffer[kAlgoLen];
int32_t spo2 = -999, heartRate = -999;
int8_t validSpo2 = 0, validHeartRate = 0;
```

These arrays feed the retained Maxim comparison calculation. Its `heartRate/spo2` variables initialize to “no result,” and the flags initialize to invalid. The transmitted estimates instead come from `ppg::estimate()` using `histRed/histIr`; the resulting `bpm` and `sat` feed the packet. The Maxim result is printed as `maxim=` for comparison only.

4.3 `system/tests/test_vitals_host.cpp` — modified estimator tests

This host C++ test already exercised Luis's estimator on synthetic signals. We added three-second windows across several known heart rates and noise seeds, a too-short two-second-window rejection check, and checks that short-window noise and flat optical contact do not become valid vitals.

The test uses an identity lookup table to examine ratio behavior; it is not a calibration study. Synthetic success does not prove accuracy on a hand, across modules, or during steering.

4.4 `system/tests/test_mux_host.cpp` — new firmware simulation

This C++ harness includes the real `.ino` sketch and supplies simulated Arduino, I2C, sensor, clock, serial, and BLE types. It exercises firmware decisions without a connected ESP32.

Checks include no-contact sleep, first-contact selection, other channels staying idle, continued streaming on the selected sensor, contact-loss failover, fresh history, single-source frame samples, overflow and timeout rejection, brief contact interruption, missing MUX, and failed shutdown verification.

The fake sensor asserts that another module is not awake when one is awakened. This is a simulated invariant, not a physical current measurement. BLE methods are stubs, so this test does not prove wireless delivery or Pi reception.

4.5 `system/tests/test_mux.py` — new test runner

This Python unittest creates temporary placeholder headers, compiles the C++ harness with `g++`, and executes it. A failed assertion or compiler error fails the test. Temporary files are automatically cleaned up. If `g++` is unavailable, this test is skipped; inspect the test summary rather than assuming every test ran.

It integrates the new harness into the existing command:

```
python3 -m unittest discover -s system/tests -v
```

4.6 `system/esp32/decode_serial_capture.py` — new capture decoder

This computer-side Python tool converts diagnostic RAW `ppg=N ...` lines into CSV. It imports Luis's existing decoder from `system/common`, preserving the original CRC and packet interpretation.

It ignores ordinary status lines, rejects malformed RAW records, and emits one row per raw sample. Columns include sensor number, sequence, frame start, sample offset, red, IR, BPM, SpO2, flags, and quality. The sensor label comes from the USB prefix; it is not added to the BLE packet.

```
python3 system/esp32/decode_serial_capture.py capture.log > samples.csv
```

A normal serial log has no RAW records and yields only the CSV header. Raw capture must be enabled in the firmware build first. This tool decodes samples; it does not independently determine clinical accuracy.

4.7 `system/esp32/README.md` — current quick-start

This file links to the current PDF and editable guide, explains the firmware folder, and provides terminal commands for flashing, monitoring, logging, and testing. It distinguishes standalone operation from optional development tools and states the remaining validation limits.

4.8 Guide source, PDF, and builder — maintained documentation

`docs/MULTI_PPG_IMPLEMENTATION_GUIDE.md` is the editable technical source. `docs/Team18_Multi_PPG_Implementation_Guide.pdf` is its navy-styled rendered edition. `docs/tools/build_multi_ppg_guide.py` produces the PDF, including its cover, linked contents, tables and page numbers. The builder runs only on a development computer; it has no role in acquisition or transport.

The previous separate implementation and bench notes have been removed after their findings were consolidated here. The root `README.md` links to this branch's current guide and quick-start. Local `/tmp` captures are temporary artifacts, not permanent repository documentation.

5. What the Pi receives and what it does not

The ESP32 retains Luis's protocol-v3 layout: 472 bytes for 100 red/IR pairs, sequence and timing fields, configured sample rate, BPM, SpO2, quality, validity/contact/range flags, and CRC-16. BLE service and characteristic UUIDs and MTU-based chunking remain the existing ones.

No MUX channel or sensor ID is added to the radio packet. The Pi sees one logical stream. Changing sensors clears local history but retains sequence continuity. During scans and failed partial frames there can be timing gaps; the firmware does not fabricate samples to fill them.

The display distinction is deliberate:

Location	Invalid value
USB serial display	-- plus a diagnostic reason
Encoded BLE packet	Numeric -999 with the original validity flags
Pi / phone	Existing decoding and invalid-data behavior

link=down means no BLE client is connected, so frames are not notified. link=up is useful but still does not prove the Pi decoded every packet. Receiver logs and matching sequences/CRCs are needed for that proof.

6. Development computer setup and optional trial tools

These commands are for firmware installation, updates, and troubleshooting. They are not steps required every time the assembled system is powered on.

The workstation was prepared with Arduino CLI 1.5.1, ESP32 Arduino core 3.3.12, and SparkFun MAX3010x library 1.1.2. The existing flash script uses the classic ESP32 FQBN with **40 MHz DIO** flash settings. These settings were retained from Luis's documented boot-stability work; we did not re-establish that earlier diagnosis here.

The firmware persists in flash and starts when the ESP32 is powered. Opening VS Code or running `tail` does not start the firmware; those actions only expose development tools or logs.

Fedora initially denied access to the CP210x serial port. Temporary access is:

```
sudo setfacl -m u:${id -un}:rw /dev/ttyUSB0
```

This grants the current user read/write access to the present device node. Unplugging removes that node and its temporary ACL. It does not affect whether firmware runs or whether the ESP32 can communicate with the Pi over BLE.

Luis's guide provides a persistent rule for the detected CP210x VID/PID. A local administrator can install it once:

```
echo 'SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", TAG+="uaccess"' | sudo tee /
etc/udev/rules.d/70-esp32-cp210x.rules
sudo udevadm control --reload-rules
```

Reconnect afterward. This rule was explained to the user; its installation has not been verified here. It grants access through the active local desktop session and applies to devices matching those IDs.

From the repository root, close any serial monitor before uploading:

```
bash system/esp32/flash_esp32.sh /dev/ttyUSB0
```

Then start a single monitor, optionally capturing a log:

```
~/local/bin/arduino-cli monitor -p /dev/ttyUSB0 -c baudrate=115200 \
2>&1 | tee -a /tmp/team18-esp32-serial.log
```

A second terminal may follow the file without opening the device again:

```
tail -f /tmp/team18-esp32-serial.log
```

Ctrl+C stops the terminal process, not the firmware. `/tmp` logs are temporary. Copy important captures to a chosen persistent location before rebooting. If the port changes, update the terminal command accordingly.

7. Reading diagnostics and collecting a waveform

Field or reason	Meaning
PPG1-PPG4	Current selected module in the serial diagnostic
WARMUP, COLLECTING_SAMPLES	Not enough uninterrupted history yet
CONTACT_UNSTABLE, CONTACT_INTERRUPTED	Contact dipped below the threshold and history was cleared
LOW_QUALITY, NO_ACCEPTED_PULSE	The estimator did not accept a pulse period/rate
SPO2_RATIO_REJECTED	A pulse was accepted but the oxygen-ratio calculation did not yield a valid lookup
TRACKING, OK	Algorithm accepted the estimate; not an accuracy certification
bpm=, spo2=	Estimates encoded for transmission
maxim=	Old comparison algorithm, serial only
q= / peak=	Accepted-period quality / largest candidate correlation; different diagnostics
history=	Nominal seconds of retained clean samples
ovf=	Reported FIFO overflow counter
win=	Time spent collecting this frame; not alone a precise sensor-clock measurement

Why the display changed from -999? to --

The original serial line used a numeric sentinel plus a validity suffix. For example, `bpm=-999? spo2=-999?` means neither result was accepted. -999 is the software's missing-result marker, not a measured negative heart rate. The question mark was added by the serial formatting expression when its validity flag was false.

The current serial formatter produces `bpm>-- spo2>--` and a reason. The actual code prints **two dashes**, not three; “---” in conversation refers to this unavailable-value display. The new strings are created only after the numeric packet is constructed and sent. Neither the original `heartRate/spo2` initialization nor the wire-format invalid values were replaced by text.

Stage	Before	Current behavior
Estimator cannot accept a result	Invalid flag; no accepted estimate	Same acceptance logic
Numeric packet field	-999	-999
Packet validity flags	Clear for the invalid estimate	Same flags

Stage	Before	Current behavior
USB screen	-999?	- - with explicit reason
Pi/phone parsing	Existing binary decoder	Same binary decoder

This is a presentation change, not an accuracy correction or a substitute for a missing measurement. An accepted result can still be wrong, as the bench review demonstrates. No last-known value is substituted to conceal a rejected window.

“Nine readings, then a result on the tenth”: observed delay versus code

The user observed repeated unavailable readings before a value appeared around the tenth reading. That observation belongs in the troubleshooting record. **The inspected Luis, Samantha and current sketches do not contain a rule that waits for exactly ten serial readings before calculating.** A result near that point can reflect sufficient history plus the first window that passes quality checks; the exact cause of an unrecorded interval cannot be reconstructed from the observation alone.

“Reading” can mean three different things:

Unit	Configured quantity	Meaning
Raw sample pair	100 per nominal second	One paired red/IR optical measurement; not one computed heartbeat
Estimator sample	25 per nominal second	Average of four raw pairs
Completed frame/status line	About one per nominal second during acquisition	A packet containing 100 raw pairs and a vitals estimate or invalid markers

Nine raw pairs represent roughly 90 ms, which is not enough for this estimator's pulse window. Nine completed frames represent roughly nine seconds of samples. Scanning, contact loss, and processing affect wall-clock timing, so these units must not be confused.

The source history shows three stages:

1. **Luis's single-sensor firmware:** `setup()` primes 100 estimator samples at 25 Hz, approximately four seconds, before entering the streaming loop. That is a four-second buffer, not a ten-frame counter.
2. **Samantha's original four-sensor firmware:** streams frames while accumulating fresh history; estimation waits until `histFilled >= kAlgoLen`, with `kAlgoLen = 100`. Early frames can contain raw data and invalid vitals.
3. **Current firmware:** the transmitted estimator is attempted at `kVitalsMinSamples = 75`, approximately three seconds. The Maxim comparison still needs 100 samples. These two calculations therefore can become available at different times.

For uninterrupted contact, the current sequence is:

First completed frame	25 history samples	WARMUP; vitals unavailable
Second completed frame	50 history samples	WARMUP; vitals unavailable
Third completed frame	75 history samples	First estimate ATTEMPT
Later frames	100 ... 200 samples	Re-estimate; history grows to 8 s
After 200 samples	Rolling 8 s history	Discard oldest as new data arrives

After the third frame, a poor waveform can still produce NO_ACCEPTED_PULSE for the fourth, ninth, tenth, or later frame. The calculation is being attempted; it is not necessarily being skipped. A contact interruption clears history and restarts warm-up. The frame counter continues across switches, so #9 means the tenth completed frame since boot, not necessarily the tenth frame of the present finger placement.

In the recorded initial test, frame #9 happened to contain an accepted estimate, but frame #3 had already contained one and frame #10 differed substantially. That recording does not support an “only calculate on the tenth reading” rule. In a subsequent clean capture, two warm-up frames were followed by 38 consecutive accepted frames. Neither example establishes measurement accuracy.

What the estimator actually checks

ppg_vitals.h removes a local baseline and lightly smooths the IR waveform, then searches plausible beat periods by normalized autocorrelation. It selects a qualifying local peak with periodicity at least 0.5 and a resulting rate within the configured 30–220 BPM range. SpO2 depends on an accepted pulse and an admissible red/IR AC-to-DC ratio lookup. Strong DC light return can establish contact while these pulse checks still fail.

The quality gate explains why optical contact and a visible LED do not guarantee a number. Its limitations also matter: an incorrect period can pass the gate. The code does not compare against a reference instrument and does not certify a reading merely by printing OK.

For optional waveform capture, close the monitor, then build and upload a separate diagnostic build:

```
~/local/bin/arduino-cli compile \
  --fqbn esp32:esp32:esp32:FlashFreq=40,FlashMode=dio \
  --build-property compiler.cpp.extra_flags=-DPPG_SERIAL_RAW=1 \
  --build-path /tmp/team18-diagnostic system/esp32/ppg_transmitter

~/local/bin/arduino-cli upload \
  --fqbn esp32:esp32:esp32:FlashFreq=40,FlashMode=dio \
  --input-dir /tmp/team18-diagnostic -p /dev/ttyUSB0 \
  system/esp32/ppg_transmitter
```

Capture serial output to capture.log, then use the decoder in section 4.6. Diagnostic printing adds USB traffic and affects loop timing; watch for overflows and analyze timing accordingly. A normal build through flash_esp32.sh disables the extra RAW output again.

8. Problems encountered, actions taken, and remaining limits

Problem	Action and outcome
Unsure whether Samantha retained Luis's architecture	Compared Git ancestry and file trees; confirmed the original extension changed only the transmitter
First-time Fedora toolchain	Installed the documented CLI, core, and library; compiled and uploaded successfully with flash-hash verification
USB permission denied after reconnect	Temporary ACL restored access; documented Luis's persistent udev option
Searching every candidate delayed selection	Changed to first passing optical contact, with a shorter idle retry pause
Old/partial data around contact loss	Clear history and discard partial frames; simulated checks verify isolation
Repeated -999 during contact	Captured waveform and replayed the same estimator; identified rejected irregular waveforms, not simply endless startup
A steadier contact produced valid output	One diagnostic segment yielded 38 consecutive accepted estimates around 78–83 BPM; demonstrates operation under that placement, not general accuracy
Large jumps still occurred in another run	Documented as unresolved. PPG1 accepted 40 then 77 BPM; a mistaken multiple of the pulse period is a hypothesis, not a proven cause
Invalid serial values were confusing	Use -- and reason strings only on USB; preserve wire sentinels and flags
HTTPS Git push lacked credentials	Used the existing working SSH key to push to the same repository and branch

Do not describe this work as “all measurement errors fixed.” Selection and diagnostics improved, but stable-looking output can still be wrong. The estimator's thresholds and calibration were not relaxed to make more numbers appear.

9. Verification and an accuracy plan

The existing Python suite plus the new MUX harness reported **21 passing tests**. Coverage includes encoding/decoding, CRC corruption detection, packet reassembly, shared-module consistency, synthetic estimator checks, and simulated sensor-selection faults. Final normal firmware compiled using approximately 1.13 MB flash and 49,164 bytes of global RAM and was uploaded with flash verification.

Hardware serial output showed all four channels responding, real contact selection and switching, and runs without reported FIFO overflows. In the reviewed 148-frame run:

Sensor/contact	Accepted BPM	Limitation
PPG4 first/later	70–79 / 72–78	Consistent intervals, but no independent reference
PPG2	30 in one contact; 41–74 in another	Rejections and questionable accepted estimates
PPG1	40–77	Adjacent accepted frames doubled from 40 to 77
PPG3	None in that run	Only one warm-up frame

No synchronized reference was recorded. Therefore no BPM error or SpO2 accuracy can honestly be calculated for those runs. No live Pi or phone receipt was verified; logs showed `link=down`.

For the next bench assessment, record an independent reference at matching times, assess each module separately, repeat placements, and retain invalid/missing results. Report average absolute BPM error, largest errors, and valid-output coverage. Reference-device averaging and our 3–8 second window affect timing comparisons. Use the repeatable comparison procedure below; its suggested engineering targets are not medical certification criteria.

Preserved observations from the superseded notes

An initial 84-frame test selected PPG4, then PPG2, then PPG1, preserving frame numbering across the switches. PPG4 produced 12 tracking frames out of 60; PPG2 produced 3 out of 20; PPG1's four-frame contact never reached tracking. This demonstrated selection and also exposed poor measurement availability.

A later diagnostic capture mirrored CRC-protected packets over USB and replayed their paired samples through the unchanged C++ estimator. One 20-frame placement yielded no accepted estimates. A steadier PPG4 placement yielded two warm-up frames, 38 consecutive accepted estimates at 78–83 BPM, and an invalid frame during removal. No ADC clipping was seen in those inspected windows. Motion, pressure and placement are possible contributors; the recording does not isolate them experimentally.

The separate 148-frame review recorded PPG2 values of 41 BPM / 88% SpO2 after interrupted contact and PPG1's 40 → 77 BPM transition. These are device outputs, not independently validated physiological events. There was no synchronized reference measurement, and no firmware change was made merely to smooth those values.

Repeatable comparison procedure

1. Record each PPG separately for 60–120 seconds with a simultaneous independent reference. Repeat each placement three times.
2. Preserve startup, rejected frames and interruptions in the record. Evaluate stationary operation first; assess controlled movement separately.
3. Align the time intervals and account for the reference's display averaging and the ESP32's 3–8 second estimation window.
4. Report mean absolute BPM error, signed bias, largest differences and valid-output coverage per module. Report SpO2 differences in percentage points.
5. Define pass/fail criteria before the comparison. A proposed team bench target is 95% of matched accepted BPM estimates within ± 5 BPM, with at least 90% valid coverage after a declared warm-up. These are provisional engineering goals, not demonstrated results or medical certification limits.

Agreement with another oximeter is only an agreement check; a stable displayed number alone is not evidence of accuracy. No reference-based error metric can be calculated retrospectively from our serial log alone.

10. Commit history and the next ecosystem phase

The implementation was committed and pushed as:

```
49918f1 Improve multi-PPG selection and add bench diagnostics
branch: samantha/multi-ppg
repository: JosselinPonce/FIU-SeniorDesign
```

Useful inspection commands:

```
git status
git show --stat 49918f1
git show --pretty="" --name-only 49918f1
git show 49918f1
```

This guide is a later documentation artifact; it is not one of the eight files in that commit.

The next phase follows Luis's `docs/SYSTEM_GUIDE.md`: inspect/setup the Pi relay, prove ESP32 packet receipt and CRC/sequence behavior, then install/configure the appropriate phone build and prove Pi → phone delivery. Finally verify local storage and authorized cloud upload. Phone developer-mode and build/signing steps depend on the actual phone platform and available developer account; those will be handled during the separate setup phase.

Do not change the Pi protocol to represent four sources: this design still sends one selected source. Test contact switching and invalid-data behavior at every receiver, and keep measurement validation separate from successful transport.

11. Source map and document maintenance

Primary project sources are Luis's supplied `Team18_System_Guide.pdf`, `docs/SYSTEM_GUIDE.md`, commits `f2c6b3c`, `e01b611`, and `49918f1`, and the current files explained in section 4. The two former multi-PPG notes have been consolidated and removed; their evidence is retained above. The summary of bench observations is drawn from the session's recorded output and the historical committed notes; it is not an independent reference measurement.

The editable source for this guide is `docs/MULTI_PPG_IMPLEMENTATION_GUIDE.md`. The companion PDF is `docs/Team18_Multi_PPG_Implementation_Guide.pdf`. The additional `docs/tools/build_multi_ppg_guide.py` renders this Markdown into the PDF; it is documentation tooling only and is separate from the eight implementation files.

Luis's system guide remains the source for downstream ecosystem setup. This guide records the ESP32 extension and its verification boundary so future work does not mistake successful selection, successful transmission, and accurate physiology for the same result.